

A Multi-Threading Algorithm for Constrained Path Optimization Problem on Road Networks

Kousik Kumar Dutta Ankita Dewan Venkata M. V. Gunturi

Indian Institute of Technology Ropar, Rupnagar, India

kousik.21csz0004@iitrpr.ac.in 2017csz0012@iitrpr.ac.in gunturi@iitrpr.ac.in

WISE 2022 | Broader search. Better routes. Multicore execution.

1 Motivation & challenge

Go beyond shortest-path routing.

Maximize road wideness or navigability while limiting total travel cost.

Find a route using wider roads while allowing only a limited detour over the shortest path.

CPO reduces to **NP-hard arc-orientteering**.

Small search: fast, but may miss good routes.

Broader search: better score opportunities, with more computation to distribute across threads.

2 CPO problem

Input: directed $G = (V, E)$, source s , destination d , and permissible overhead.

Each edge has **cost** > 0 and **score** ≥ 0 .

Path metrics: $\Gamma(P)$ is total collected score; $\Phi(P)$ is total consumed cost.

$$\max_{P^*: s \rightarrow d} \Gamma(P^*)$$

subject to $\Phi(P^*) \leq budget$

$$budget = \Phi(P_{\text{shortest}}) + overhead$$

Output: a directed $s-d$ path within budget.

Experimental setup

Three real-world road networks

Network	Nodes	Edges
Delhi	52,576	150,488
Buenos Aires	263,783	864,408
London	285,050	749,382

Synthetic scores: random edges receive random scores 1–15; all others have score 0.

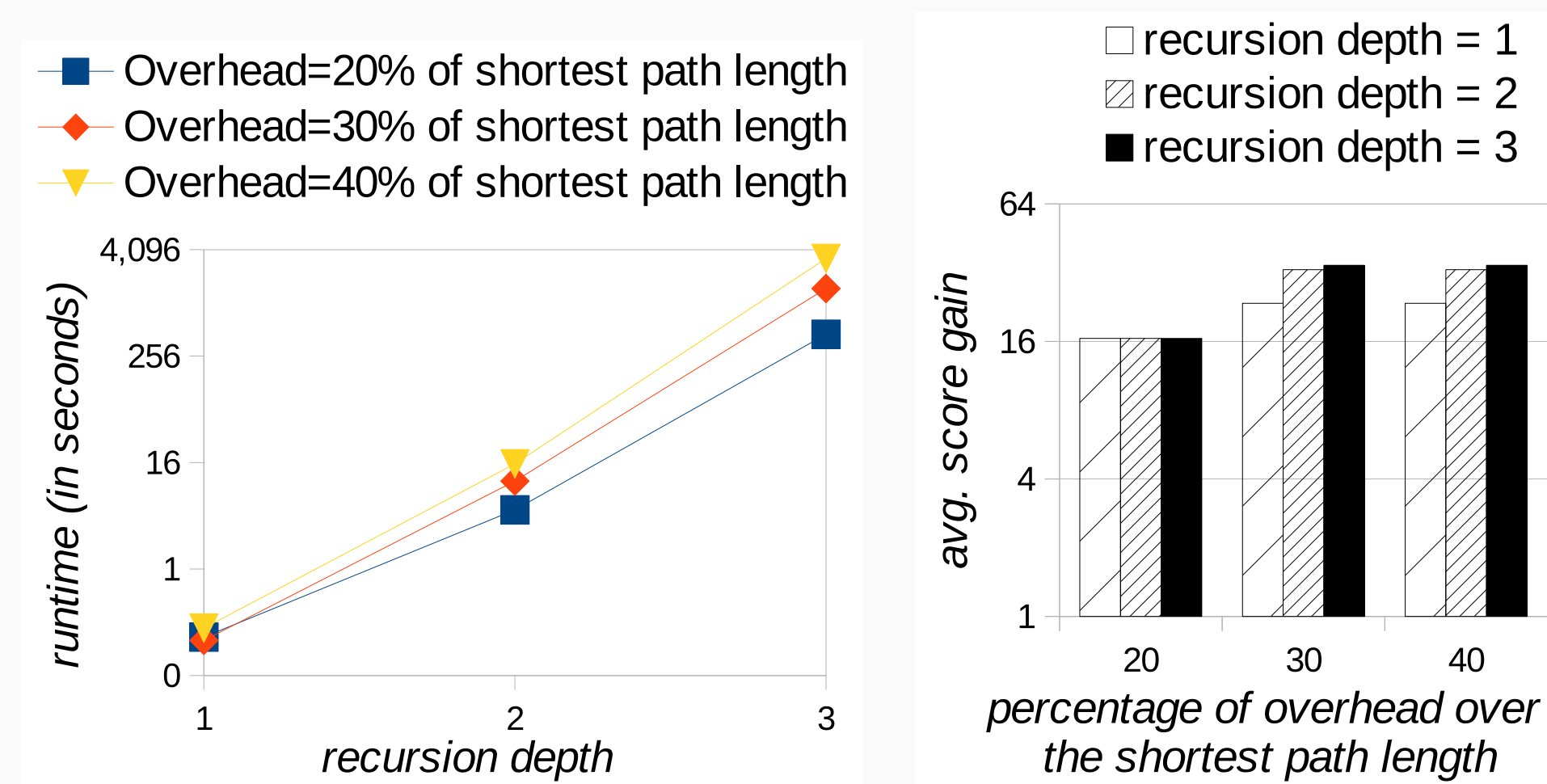
Java 11 on Ubuntu; Intel Xeon Platinum 8280M; **128-thread capacity**; **2048 GB RAM**. Base 2.70 GHz; maximum boost 4.00 GHz.

Metrics: execution time; score gain = returned-path score – shortest-path score.

Queries: 100 pairs per path-length range; length counts shortest-path edges.

Sensitivity / design choices

Depth costs much more than it gains.



Choose $\theta = 1$: runtime rises exponentially with depth for only small score gains.

Depths 1, 2, 3 on smaller Delhi: 9,401 nodes, 25,941 edges; 40% positive-score edges. Log₄ y-axes; original overhead labels retained.

Budget / density: London studies at 30/40/50% show that increasing either raises runtime and solution quality.

Selected references

[1] C. Chekuri et al. *A Recursive Greedy Algorithm for Walks in Directed Graphs*. IEEE FOCS, pp. 245–253 (2005).

[2] R. Kaur et al. *Finding the Most Navigable Path in Road Networks: A Summary of Results*. DEXA (2018). [MSWBS]

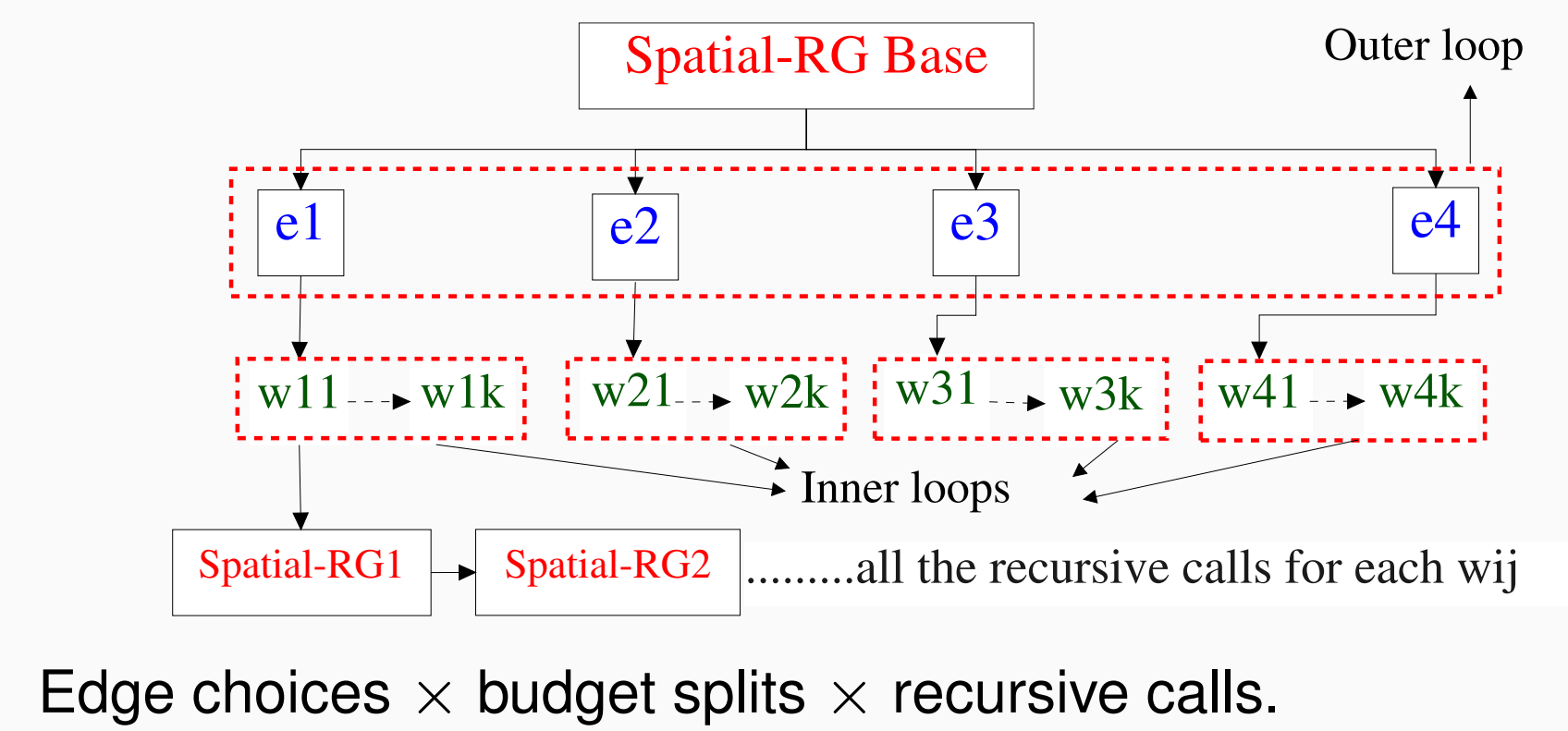
[3] Y. Lu and C. Shahabi. *An Arc Orienteering Algorithm to Find the Most Scenic Path on a Large-Scale Road Network*. ACM SIGSPATIAL, article 46 (2015). [ILS*(CEI)]

Indian Institute of Technology Ropar

WISE 2022

3 Spatial-RG: expanding the search space [1]

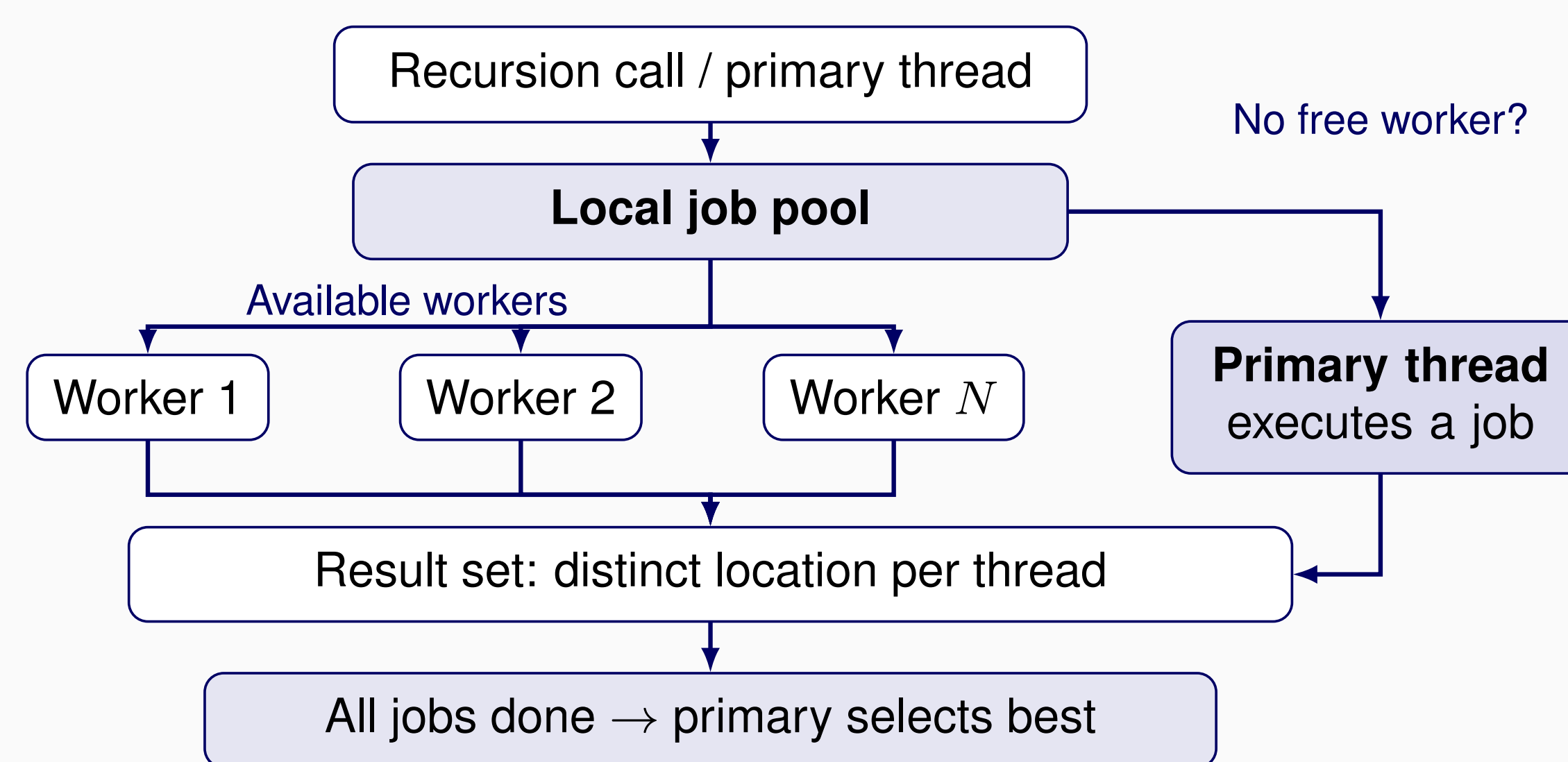
1. **Seed** with a minimum-cost path.
2. **Filter** scored candidate edges spatially.
3. **Recurse** on left/right paths and budget splits.
4. **Retain** the best feasible replacement found.



Ellipse-based spatial pruning + uniform-grid indexing

Shortest-path implementation: A* with a Euclidean-distance heuristic.

4 Parallel-Spatial-RG: keep available threads working



Dynamic assignment

Free workers take pending jobs from each local pool.

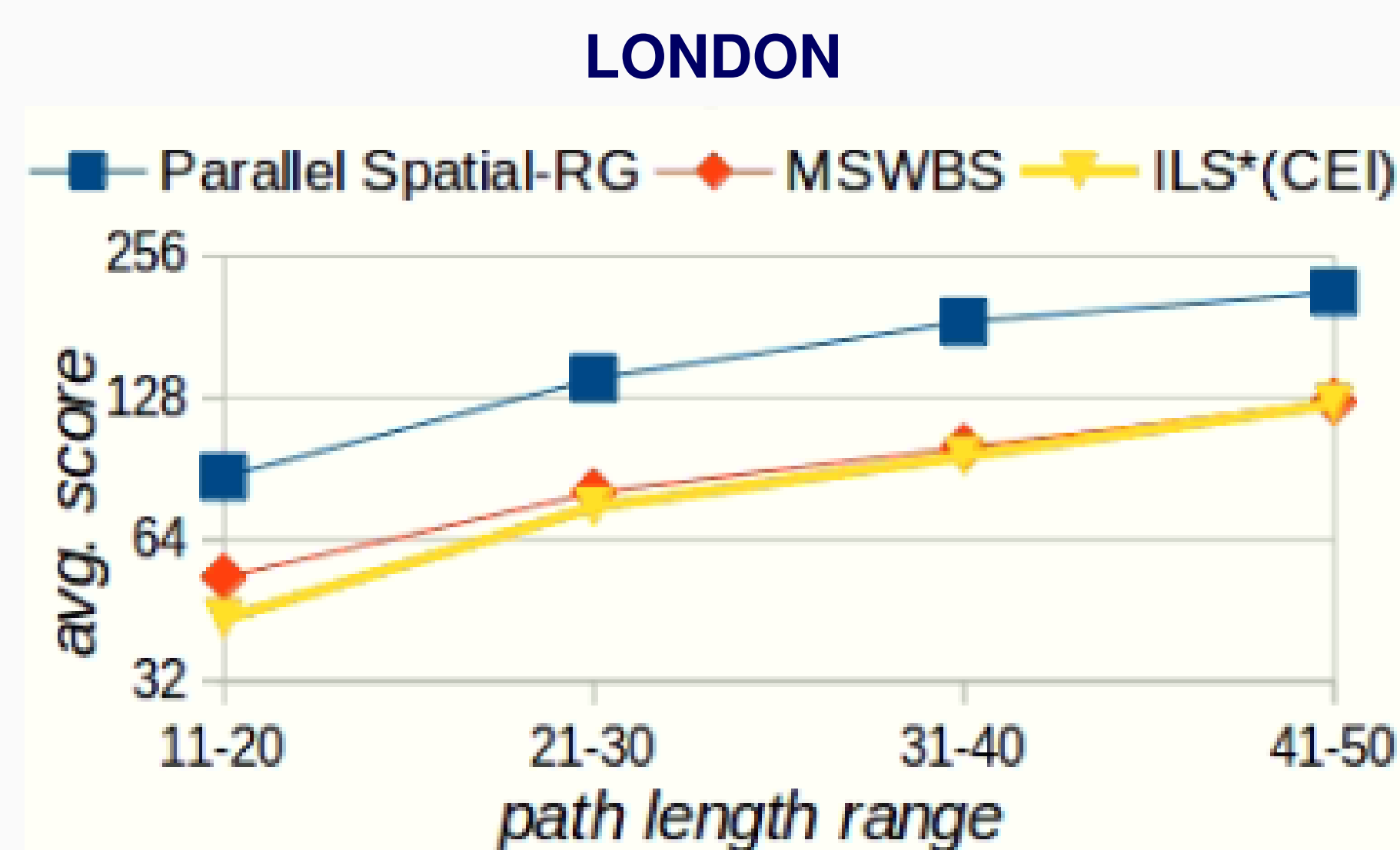
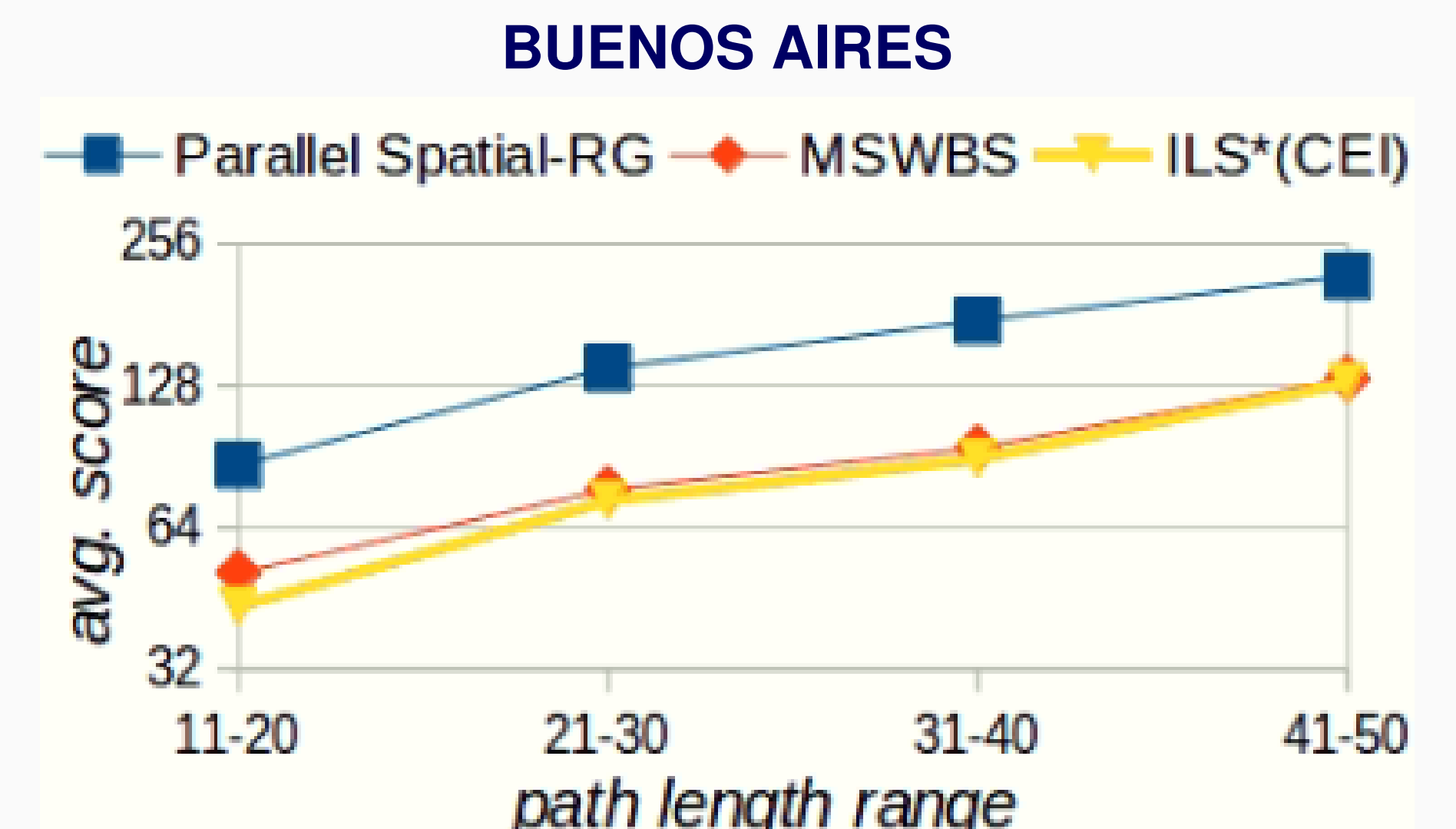
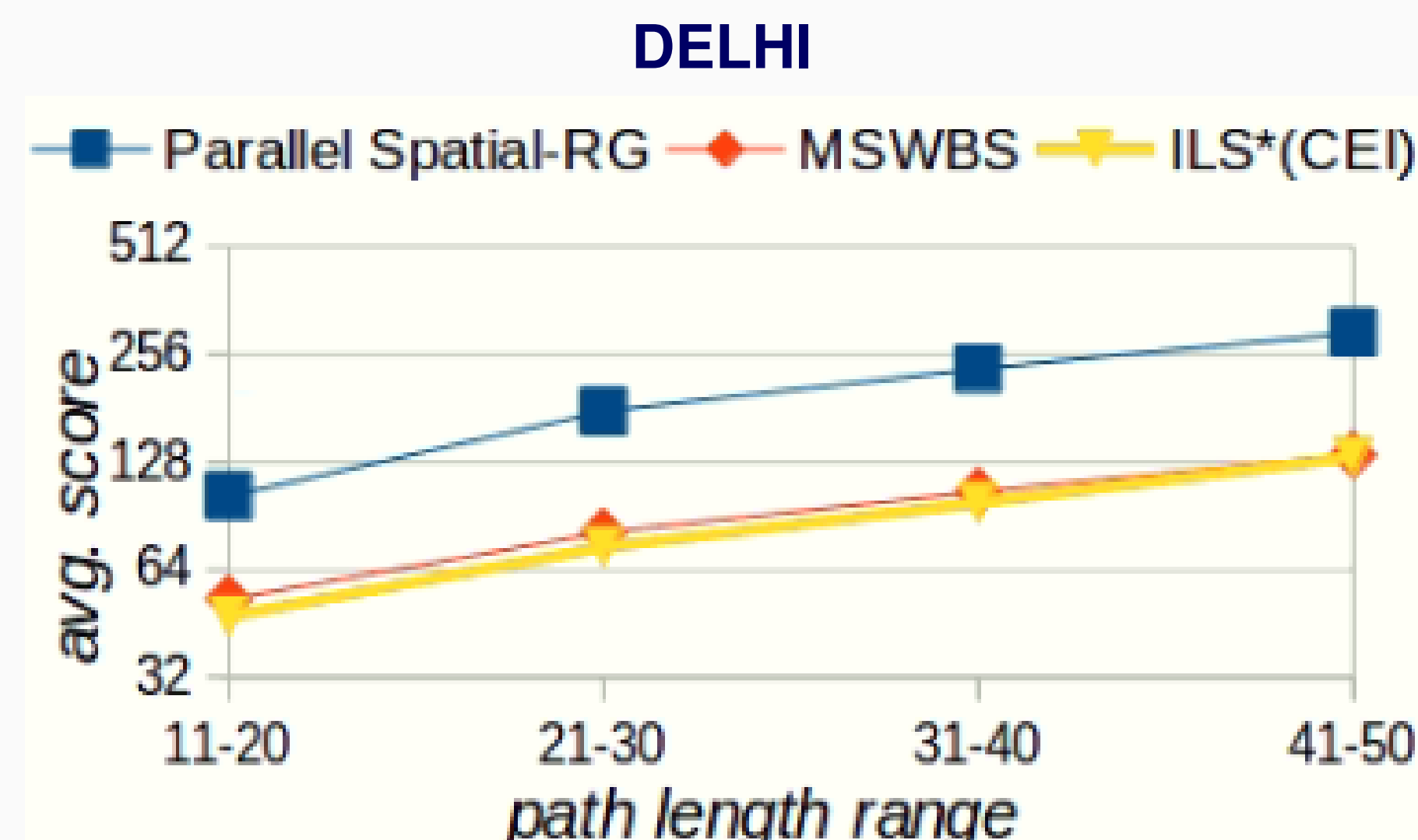
Primary-thread fallback

When workers are busy, the primary executes pending jobs.

Separate result writes

Distinct locations avoid a critical section for result writes.

5 Better solution quality across all three networks



Substantially higher average score

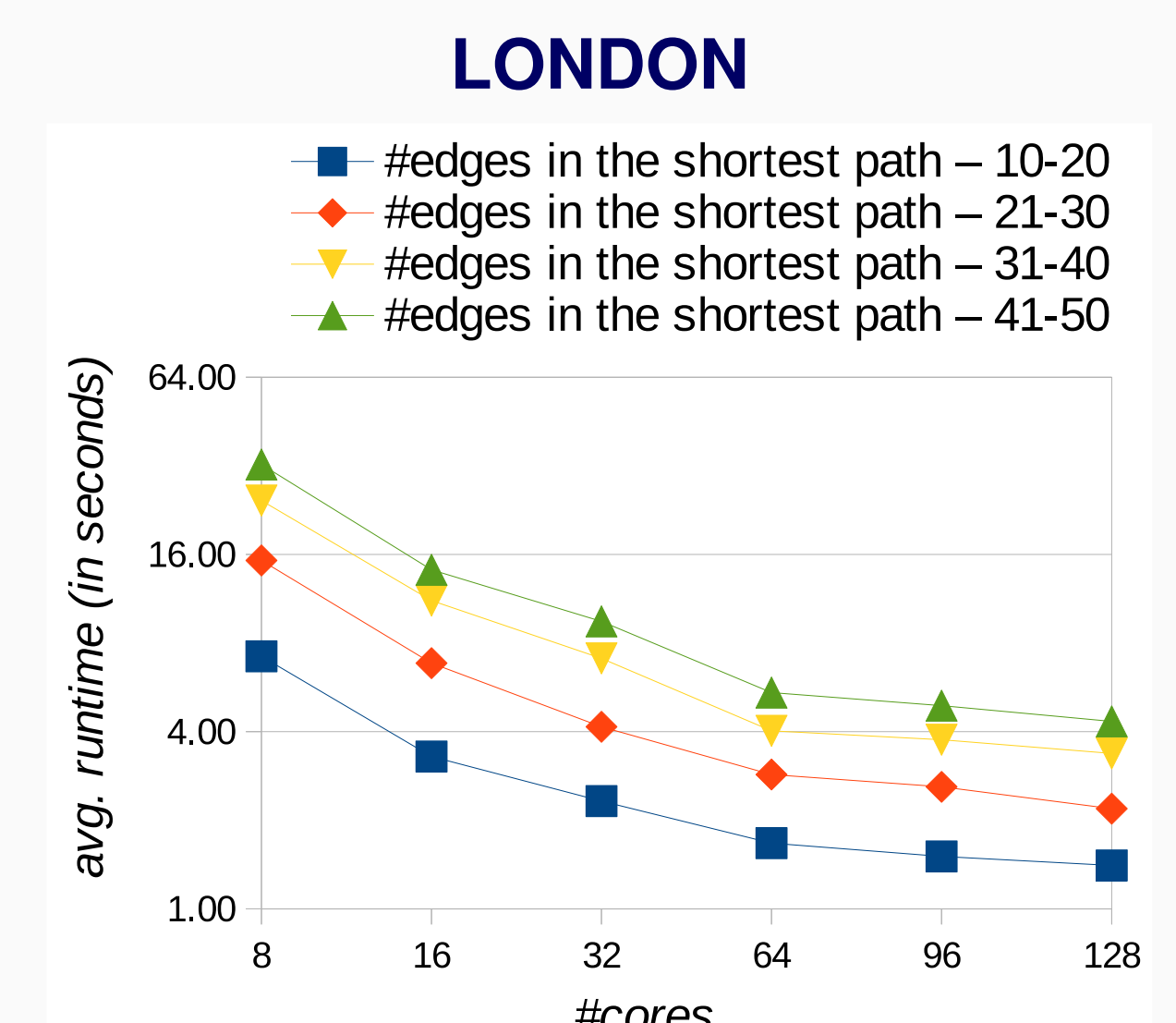
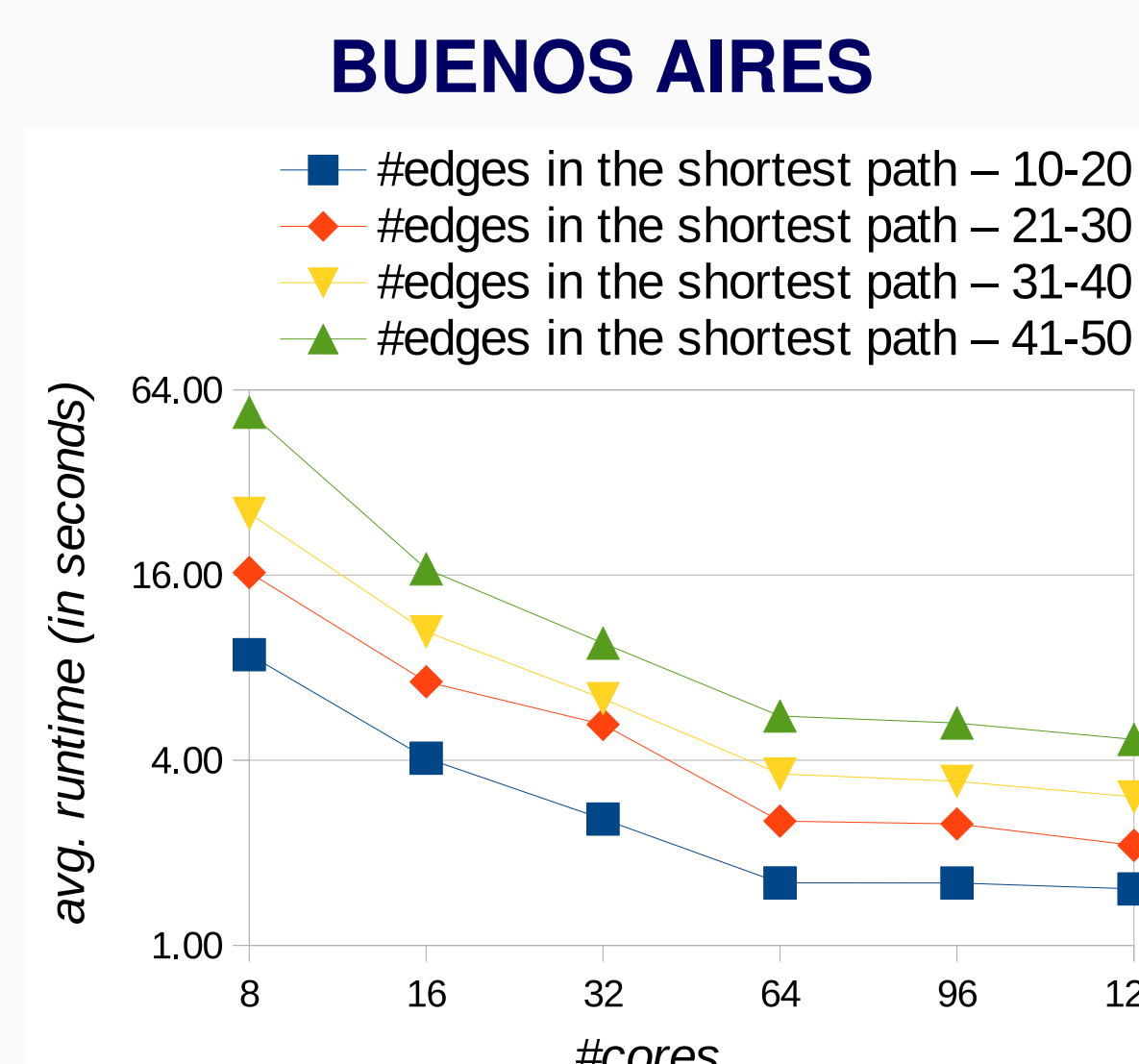
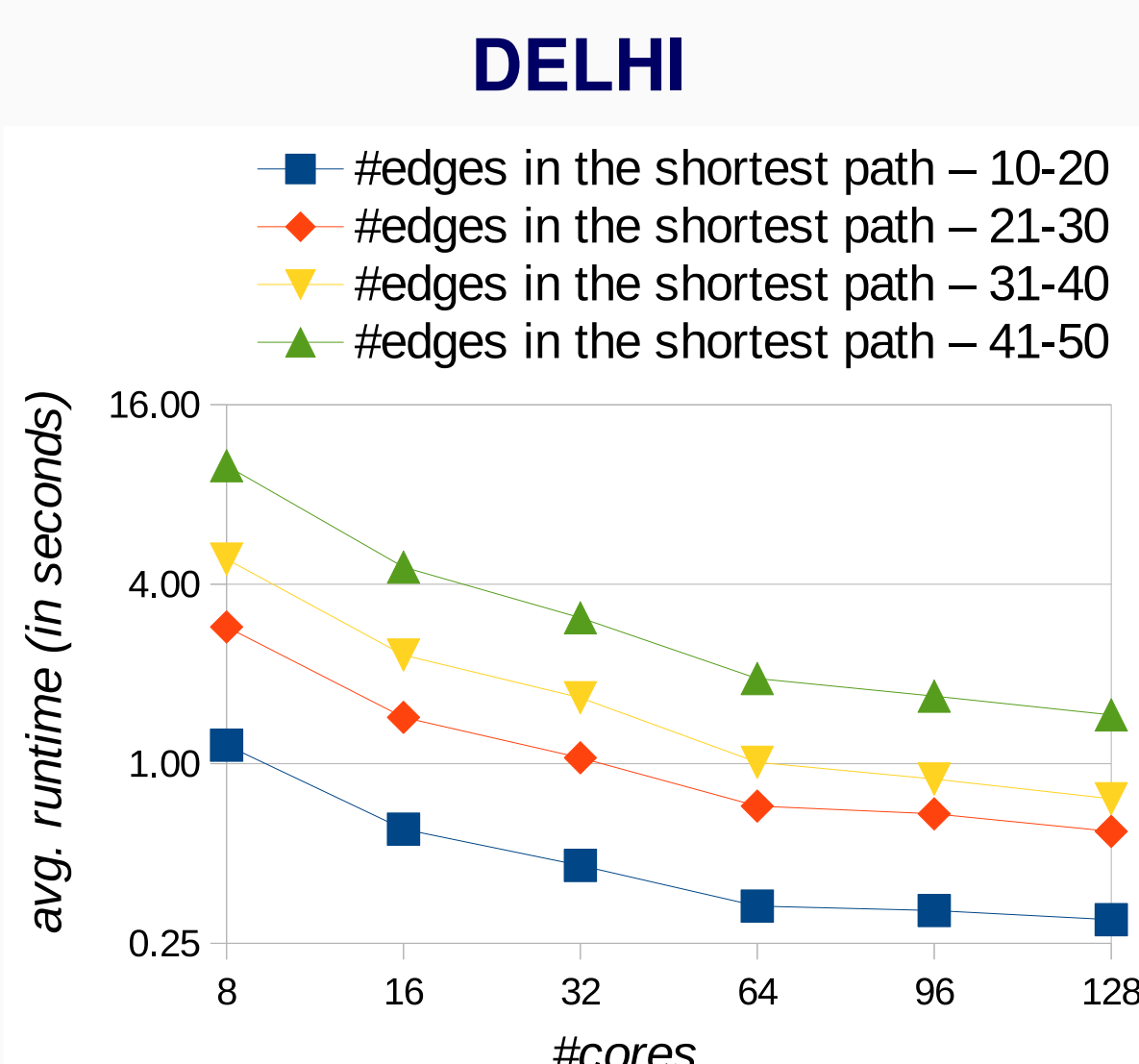
Parallel-Spatial-RG consistently exceeds **MSWBS [2]** and **ILS*(CEI) [3]** in the reported experiments.

Shared setting

30% overhead; 40% positive-score edges; $\theta = 1$. Parallel-Spatial-RG uses 128 threads.

Original plots show **average score**, with log₂ y-axes. ILS*(CEI) uses a 3 s time limit.

6 Multicore scalability: almost linear scale-up



8, 16, 32, 64, 96, 128 threads/cores; 30% overhead, 40% positive-score edges, $\theta = 1$. Log₄ y-axes. Short workloads saturate as parallel work runs out.

Varying-core study, reported averages across path lengths / maxima (s): Delhi 0.81 / 1.32; Buenos Aires 2.7 / 4.2; London 2.9 / 4.15.

Takeaways

Larger search

Broader search improves reported route scores.

Practical parallelism

Local pools + primary fallback share pending work.

Multicore scaling

Almost linear scale-up in the reported experiments.